Model Evaluation and Selection

Page No:
Date.

Unit - 5

1] Confusion Matrix → 2X2

Actual Label	Predicted label
Positive	Positive
Positive	Positive
Positive	Negative
Positive	Positive
Negative	Negative
Negative	Positive
Negative	Negative
Negative	Negative
Positive	Positive
Negative	Negative

→ Confusion Matrix based on this →

		Predicted: Positive	Predicted: Negative
Actual ?	Positive	4	1
	Negative	1	4

Since, True Positive (TP) = 4
False Negative (FN) = 1
False Positive (FP) = 1
True Negative (TN) = 4

1] Accuracy →

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} = \frac{4+4}{4+4+1+1}$$

$$\text{Accuracy} = \frac{8}{10} = 0.8 \quad (80.0\%)$$

Scanned with CamScanner

2) Precision →

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{4}{4+1} = \frac{4}{5} = 0.8$$

3) ~~Rec~~ Recall →

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{4}{4+1} = \frac{4}{5} = 0.8$$

4) F1 - Score →

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$= 2 \times \frac{0.64}{1.6} = 0.4 \times 2 = 0.8$$

3X3

25 samples

		Predicted A	B	C
Actual	A	5	2	1
	B	1	6	2
	C	0	1	7

Conversion →

Class 1: A →

		Predicted A	B&C
Actual	A	5 (TP)	3 (FN)
	B&C	1 (FP)	16 (TN)

TP = 5, FN = 2+1 = 3

FP = 1, TN = 16

Scanned with CamScanner

Class B →

Actual		Predicted B	Predicted A&C
	B	6 (TP)	3 (FN)
	A&C	3 (FP)	13 (TN)

TP = 6 , FN = 3 , FP = 3 , TN = 13

Class C →

Actual		Predicted C	Predicted A&B
	C	7	1
	A&B	3	14

TP = 7 , FN = 1 , FP = 3 , TN = 14

Now, calculate Metrices Per class →

1] For class A →

- Accuracy = $\frac{5+16}{25} = 0.84$
- Precision = $\frac{5}{5+1} = 0.833$
- Recall = $\frac{5}{5+3} = 0.625$
- $F_1 = \frac{2 \times (0.833 \times 0.625)}{(0.833 + 0.625)} = 0.714$

2) For class B →

- Accuracy = $\frac{6+13}{25} = 0.76$
- Precision = 0.667
- Recall = 0.667
- F_1 = 0.667

Scanned with CamScanner

3) Class C :

- Accuracy = 0.84
- Precision = 0.700
- Recall = 0.825
- F_1 = 0.778

* Macro F1 score →

It is average of all F_1 scores →

$$\text{Macro F1} = \frac{0.714 + 0.667 + 0.778}{3}$$

$$\text{Macro F1} = 0.72$$

* Weighted F1 score →

$$\text{Weighted F1} = \frac{\sum_{i=1}^{n} F1_i \times \text{support}_i}{\sum_{i=1}^{n} \text{support}_i}$$

Where, ↗ values in rows

- support_i → No. of true instances of class i
- n → is the number of classes
- $F1_i$ → F1 score of class i

∴ For class A → $\text{support}_A = 5+2+1 = 8$

—o— B → $\text{support}_B = 1+6+2 = 9$

—o— C → $\text{support}_C = 0+1+7 = 8$

$$\therefore \text{Weighted F1} = \frac{8 \times 0.714 + 9 \times 0.667 + 8 \times 0.778}{25}$$

$$= \frac{17.939}{25}$$

$$\therefore \boxed{\text{Weighted F1} = 0.7176}$$

Scanned with CamScanner

* Cross Validation →

- Cross validation is a model evaluation techniqu used to check how well a machine learning model will generalize to an independent dataset
- Instead of training your model once on one train spilt, cross validation splits the data into mul parts and tests the model multiple times - each ti with different data.

* Diff betn Overfitting vs. Underfitting

Overfitting	Underfitting
1] Model learns too much.	1) Model learns very too li
2) To complex model	2) To simple model
3) Good training data, bad testing data	3) Good testing data, bad training data
4) Accuracy is High	4) Accuracy is Low.
5) e.g. Model depends on All particular variables and learn them very fastly	5) e.g. Model depends on only some features and also it doesnot give proper predictio

* Types of Cross validation →

1] Holdout Cross Validation →

- Here you have to split dataset into two p
  - Training set → used to train the model
  - Testing set → used to evaluate the model perfo

Scanned with CamScanner

Step 1: Take a data like 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Step 2: split it is 80 : 20

80% Training → 1, 2, 3, 4, 5, 6, 7, 8

20% Test → 9, 10

Step 3: Model automatically learn and predict & gives accuracy.

2] K-fold Cross Validation →

- In K-fold Cross validation, the dataset is divided into k equal parts ("folds").
- The model is trained and evaluated k times each time using a different fold as the test set, and the remaining k-1 folds as the training set.
  - k^{th} as test set
  - k-1 as train set

* steps →

1] split the dataset into k equally size folds.

2] For each of k iteration:
- Use 1 fold as the test
- Use k-1 as train
- Train & evaluate model

3] Average the evaluation for all k.

e.g. →

Sample	Feature (X)	class (y)
1	2.1	0
2	1.8	1
3	2.5	0
4	3.0	1
5	2.7	0
6	3.2	1
7	2.9	0

Scanned with CamScanner

Make 3 Folds:

Fold 1 : 1, 2, 3

Fold 2 : 4, 5

Fold 3 : 6, 7

For iteration 1 →

Train → Fold 1

Test → Fold 2 + Fold 3

∴ Accuracy = 80 %

For iteration 2 →

Train → Fold 2

Test → Fold 1 + Fold 3

∴ Accuracy = 90 %

For iteration 3 →

Train → Fold 3

Test → Fold 1 + Fold 2

∴ Accuracy = 100 %

∴

$$\text{Overall Accuray} = \frac{90+80+100}{3} = \frac{270}{3} = 90\%$$

* Oversampling and Undersampling →

Under

1) ~~Over~~sampling →
   - Reduce majority class instances
   - It is faster, few data points
   - May lose info, & also cause underfitting

2) Oversampling →
   - Increase no. of instances of minority class.
   - No data loss
   - model can learn more from minority class
   - Risk of overfitting, large dataset are used.

SVM

SMOTE is soln

Scanned with CamScanner

3) Stratified K-fold techiques

- To split data into k-folds while maintaining the same class distribution in each fold.

- e.g.

we take dataset of 15 values consists 3 classes

step1: A, B, C → classes with sample values

- class A - 5 samples
- class B - 5 samples
- class C - 5 samples

step2: Divide each class into 3 folds

- Here we have to place 1 or 2 samples of each class in each fold.

A → 1 or 2 per fold.

B → 1 or 2 per fold.

C → 1 or 2 per fold.

step3: Create 3 stratified folds -

1) Fold 1 → (diff. values each time but of same class)

A - 2 values, B → 2-value, C - 2 values

∴ Train → Fold 1

Test → Fold 2 & 3

2) Fold 2:

A - 2values, B → 2 values, C - 2values

∴ Train → Fold 2

Test → fold 1 & Fold 3

3) Fold 3:

A - 2 val, B → 2 val, C → 2 values

∴ Train - fold 3

∴ Test - Fold 1 & 2

Scanned with CamScanner

* Advantages →
- Maintains class balance in every fold
- Prevents bias in evaluation, especially for imbalanced data
- More accuracy & reliability.

4) Leave-One-Out-Cross-validation (LOOCV) →
- It is a special case of k-fold cross-validation, where the number of folds k equals the no. of data points

* steps →
1] For a dataset with n samples, LOOCV perform n iterations
2) In each iteration;
- use n-1 sample train
- remaining 1 as test.
3) Repeat this process n times for diff. samples
4) finally find accuracy or averages

* When to use?
- small datasets
- when you want more training data
- It has high variance

e.g. ↓

Index	Feature X	Label Y
1	2	A
2	3	B
3	4	A
4	5	B
5	6	A

for i=1
Train - 2, 3, 4, 5
Test - 1
like this for all .....

Scanned with CamScanner

Page No:
Date.

* Advantages: - Utilizes nearly all data
- Unbiased estimation of model performance
* Disadvantages: - very slow for large dataset.
- High variance since test is small.

5) Leave - P - Out - cross Validation -
- It is a generalization of LOOCV
- Instead of leaving 1 out, you have to leave p samples
- All possible combinations of p samples are used as test sets.
- The remaining n-p samples are used for testing

e.g. for similar dataset we take $p=2$
→ so $5C_2 = 10$ combination
means need 10 iterations
train → 2 values. } Make 10 combos.
test → 3 values

* Advantages -	* Disadvantages -
- Exhaustive	- Time taking if
- Better evaluation on small dt.	large n or large p.

Parameter Tunning and Optimization →

- Parameter Tunning is the process of selecting the best values for a machine learning model's parameters to improve its performance.
  - Parameters : set before training
  - Optimization : systematically searching for the best combination of these parameters.

Scanned with CamScanner

Page No:
Date.

- Parameter tunning is the process of selecting the most suitable hyperparameters for a machine learning model to maximize its performance on unseen data.
  - Hyperparameters are parameters set before training the model.
  - Unlike model parameters, hyperparameters are external settings that guide the learning process.

AI

* Why to use?
- A poorly tuned model may overfit or underfit
- Proper tuning helps:
  - Maximize accuracy or minimize error.
  - Generalizes well to new data.
  - Improve robustness and performance.

* Diff. Between Model Parameters & Hyperparameters →

Feature	Model Parameters	Hyperparameters
set by	Learning algorithm	Manually or using tunning
Learned from data	Yes	NO
e.g.	Weights in linear Regression	k in KNN, C in SVM
Tuned using	Gradient descent, Backpropagation	Grid search, Random search.

Scanned with CamScanner

* Common Hyperparameters →

Algo	key Hyperparameters
• K-Nearest Neighbours	n_neighbours, distance_metric
• Decision Tree	max_depth, min_samples_split
• SVM	C, kernel, gamma
• Random Forest	n_estimators, max_features
• XGBoost	learning_rate, max_depth.

A] Manual Search →

Manual search is the simplest method of hyperparameter tunning. It involves:

- Manually selecting different hyperparameter values.
- Training the model with each set.
- Comparing performance metrices
- choosing the best-performing configuration.

* When to use Manual search

- when dataset is small.
- When the model has few hyperparameters.
- When you understand the model well.
- For intial exploration before automated tuning.

* Advantages →

- Easy to understand & implement.
- Good for simple model or small datasets.
- Helps build intution for model behaviour.

* Disadvantages →

- Not scalable for large hyperparameter spaces.
- Can be inefficient and time consuming.
- Might miss the optimal combination.

Scanned with CamScanner

Page No.
Date.

* Grid Search →
- Grid Search is an exhaustive hyperparameter tuning techinque that:
  - Define a grid of hyperparameter values,
  - Trains a model for every possible combination,
  - Evaluates performance (usually with cross-validation)
  - Selects the best-performing combination.

* When to use Grid search?
- when you have a limited and known range of values.
- when accuracy is more important than speed.
- Ideal for model benchmarking

* How it works →
Step 1: Choose model → SVM
Step 2: Identify hyperparameters to tune
- C , - kernel , - gamma
Step 3: Create parameter grid.
Total combination = 3×2×2 = 12 models
e.g [C → [0.1, 1, 10] , kernel → ['linear', 'rbf'], gamma → [0.01, 0
Step 4: Train each combination using cross-validation →
Step 5: ~~calculate~~ select the best performing combination.

* Advantages →
- simple and through
- Guarantees finding the best result

* Disadvantages →
- Computationally expensive for large grids.
- May include redundant combinations.
- Can be slow with many hyperparameters or large datasets.

Scanned with CamScanner

Page No:
Date:

* Random Search →

- Random search is a hyperparameter tunning techinque that:
  - Samples random combinations of hyperparameter values from defined ranges
  - Trains and evaluates a model on only a subset of all possible combination.
  - Selects the combination that gives the best performance.

* Why Use Random Search?

- Useful when the hyperparameter space is large.
- Often finds good results faster than Grid search.
- Reduces computational cost by trying only a few combination

* How it works (steps):

Step1 : Choose the model:
Random Forest

Step2: Define a range:

```
param_dist = {
    'n_estimators' : [50, 100, 200, 300],
    'max_depth' : [None, 10, 20, 30],
    'min_samples_split' : [2, 4, 6],
    'criterion' : ['gini', 'entropy']
}
```

Step3: choose how many random combination to try.
Try 10 random combination out of $4 \times 4 \times 3 \times 2 = 96$

Step4: Evaluate each combination (e.g. using cross-validation)

Step5: Select the best performing hyperparameter set.

Scanned with CamScanner

* Advantages →
- More efficient than grid search for high-dimensional spaces.
- Can discover unexpectedly good combinations.
- Faster than trying all options.

* Disadvantages →
- Might miss the optimal combination.
- Results vary each run.

* AUC-ROC Curves →

• AUC →

- AUC (Area under the curve) measures the entire two dimensional area underneath the entire ROC curve.
- It gives a single scalar value to compare models.
  - AUC = 1 → Perfect model
  - AUC = 0.5 → Random guessing
  - AUC < 0.5 → Worse than random

• ROC →

- ROC (Receiver Operating Characteristic) Curve is a graphical plot that shows the performance of a binary classifier as its threshold is varied.

It plots →

• True Positive Rate (TPR) →

$$\frac{TP}{TP+FN} \quad \text{(All positive)}$$

• False Positive Rate (FPR) →

$$\frac{FP}{FP+TN} \quad \text{(All Negative)}$$

* TPR → Y-axis
* FPR → X-axis

Scanned with CamScanner

Page No:
Date:

* Why use ROC - AUC?

- Independent of threshold: It evaluates model performance at all classification thresholds.
- Balanced metric: Useful when dealing with imbalanced classes.
- Helps visualize trade-offs between sensitivity & specificity.

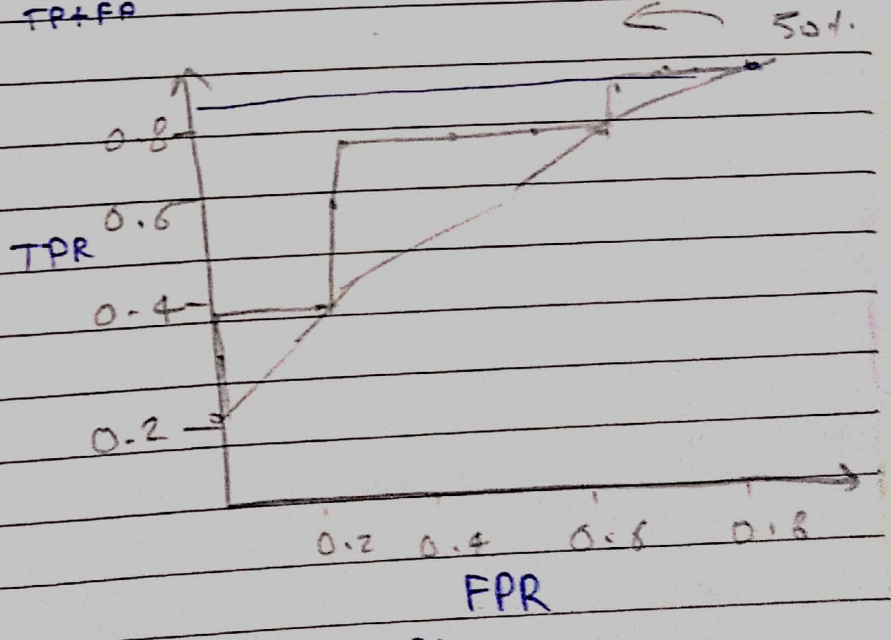
* e.g →

Class

Sample	Actual	Predict	Probability	TP	FP
1	P	P	0.90	1	0
2	P	P	0.80	2	0
3	P	N	0.70	2	1
4	P	P	0.60	3	1
5	P	P	0.55	4	1
6	P	N	0.54	4	2
7	P	N	0.53	4	3
8	P	N	0.51	4	4
9	P	P	0.50	5	4
10	P	N	0.40	5	5
				TP=5	FP=5

$\frac{TP}{TP+FP}$	TPR	FPR
0.2		0
0.4		0
0.4		0.2
0.6		0.2
0.8		0.2
0.8		0.4
0.8		0.6
0.8		0.8
1		0.8
1		1

$\frac{FP}{FP+FP}$



AUC is 0.83.

Scanned with CamScanner